

**BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**



**BÁO CÁO
ĐỒ ÁN MÔN HỌC**

**ĐỀ TÀI:
HUẤN LUYỆN MÔ HÌNH DỰ ĐOÁN BỆNH TIỂU ĐƯỜNG**

Môn học: Nhập Môn Trí Tuệ Nhân Tạo

Giảng viên hướng dẫn: Nguyễn Thị Tuyết Hải

Thực hiện bởi nhóm sinh viên, bao gồm:

1. Nguyễn Ngọc Phương	N23DCCN047	Trưởng nhóm
2. Nguyễn Trọng Nhân	N23DCCN178	Thành viên
3. Tô Hoàng Phi	N23DCCN181	Thành viên

TP.HCM, tháng 05 /2026

MỤC LỤC

MỤC LỤC.....	2
DANH SÁCH HÌNH, BẢNG.....	4
TÓM TẮT.....	5
BẢNG PHÂN CÔNG CÔNG VIỆC TRONG NHÓM.....	6
CHƯƠNG I. GIỚI THIỆU ĐỀ TÀI.....	7
1.1. Lý do chọn đề tài.....	7
1.2. Mục tiêu của đề tài.....	7
1.3. Phạm vi và đối tượng nghiên cứu.....	7
1.4. Phương pháp thực hiện.....	7
1.5. Cấu trúc báo cáo.....	8
CHƯƠNG II. CƠ SỞ LÝ THUYẾT.....	9
2.1. Tổng quan về trí tuệ nhân tạo và học máy.....	9
2.2. Bài toán phân loại trong học máy.....	9
2.3. Bộ dữ liệu sử dụng trong đề tài.....	9
2.4. Tiền xử lý dữ liệu.....	10
2.4.1. Xử lý giá trị thiếu bằng trung vị.....	10
2.4.2. Xử lý giá trị thiếu theo nhóm Outcome.....	10
2.5. Chuẩn hoá dữ liệu (StandardScaler).....	10
2.6. Thuật toán K-Nearest Neighbors (KNN).....	11
2.7. Thuật toán Logistic Regression.....	11
2.8. Thuật toán Decision Tree.....	11
2.9. Tối ưu tham số bằng GridSearchCV.....	11
2.10. Các chỉ số đánh giá mô hình.....	12
CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	13
3.1. Mục tiêu thiết kế hệ thống.....	13
3.2. Kiến trúc tổng quát của hệ thống.....	13
3.3. Thiết kế dữ liệu đầu vào.....	14
3.4. Thiết kế các phiên bản dữ liệu.....	15
3.5. Thiết kế chia tập dữ liệu.....	15
3.6. Thiết kế quy trình tiền xử lý dữ liệu.....	16
3.7. Thiết kế bước chuẩn hoá dữ liệu.....	16
3.8. Thiết kế chia tập dữ liệu.....	17
3.9. Thiết kế mô hình huấn luyện.....	18
3.9.1. Mô hình K-Nearest Neighbors.....	18
3.9.2. Mô hình Logistic Regression.....	18
3.9.3. Mô hình Decision Tree.....	18
3.10. Thiết kế tối ưu tham số.....	19
3.11. Thiết kế đánh giá mô hình.....	19
3.12. Thiết kế đầu ra của hệ thống.....	20
3.12.1. Kết quả dự đoán.....	20

3.12.2. Kết quả đánh giá mô hình.....	20
CHƯƠNG IV. TRIỂN KHAI HỆ THỐNG.....	22
4.1. Môi trường triển khai.....	22
4.2. Đọc và chia tập dữ liệu.....	22
4.3. Chuẩn hoá dữ liệu.....	23
4.4. Triển khai mô hình Logistic Regression.....	24
4.5. Tối ưu mô hình Logistic Regression bằng GridSearchCV.....	24
4.6. Triển khai mô hình Decision Tree.....	25
4.7. Tối ưu mô hình Decision Tree bằng GridSearchCV.....	26
4.8. Triển khai mô hình KNN.....	27
4.9. Tối ưu mô hình KNN bằng GridSearchCV.....	28
CHƯƠNG V. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ.....	31
5.1. Mục tiêu thực nghiệm.....	31
5.2. Kết quả huấn luyện mô hình Logistic Regression.....	31
5.3. Kết quả huấn luyện mô hình Decision Tree.....	31
5.4. Kết quả huấn luyện mô hình KNN.....	31
5.5. Bảng so sánh kết quả các mô hình.....	31
5.5.1. Bộ dữ liệu clean_diabetes.csv.....	31
5.5.2. Bộ dữ liệu diabetes_optimal.csv.....	31
5.6. Nhận xét kết quả thực nghiệm.....	32
CHƯƠNG VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	33
6.1. Kết luận.....	33
6.2. Hướng phát triển trong tương lai.....	33
TÀI LIỆU THAM KHẢO.....	34

TÓM TẮT

Đề tài "Huấn luyện mô hình AI dự đoán bệnh Diabetes" tập trung xây dựng và đánh giá các mô hình học máy có khả năng dự đoán nguy cơ mắc bệnh tiểu đường dựa trên các chỉ số sức khỏe của bệnh nhân. Bộ dữ liệu sử dụng gồm 768 mẫu với 8 đặc trưng y tế đầu vào và một biến mục tiêu phân loại nhị phân (Outcome).

Hệ thống thực hiện đầy đủ quy trình học máy bao gồm: tiền xử lý dữ liệu, chuẩn hoá, chia tập huấn luyện và kiểm tra, huấn luyện ba mô hình (K-Nearest Neighbors, Logistic Regression, Decision Tree), tối ưu siêu tham số bằng GridSearchCV và đánh giá bằng các chỉ số Accuracy, Precision, Recall, F1-score.

Kết quả thực nghiệm cho thấy mô hình KNN đạt hiệu suất tốt nhất trên bộ dữ liệu diabetes_optimal.csv với độ chính xác 85.06%, cho thấy việc xử lý missing value theo nhóm Outcome mang lại hiệu quả rõ rệt so với thay thế bằng trung vị chung.

BẢNG PHÂN CÔNG CÔNG VIỆC TRONG NHÓM

STT	MSSV	HỌ VÀ TÊN	NHIỆM VỤ
1	N23DCCN047	Nguyễn Ngọc Phương	Tìm kiếm dataset, xây dựng mô hình KNN, làm báo cáo chương I, II, III, IV, V
2	N23DCCN178	Nguyễn Trọng Nhân	Xây dựng mô hình Logistic Regression, làm slide thuyết trình
3	N23DCCN181	Tô Hoàng Phi	Xây dựng mô hình Decision Tree, làm báo cáo chương VI, tài liệu tham khảo, chỉnh sửa format báo cáo

Link Github: https://github.com/mynameisFuong/final_project_AI

CHƯƠNG I. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Bệnh tiểu đường (diabetes) là một bệnh rối loạn chuyển hóa mạn tính liên quan đến sự bất thường trong quá trình điều hòa đường huyết của cơ thể. Nếu không được phát hiện và kiểm soát kịp thời, bệnh có thể dẫn đến nhiều biến chứng nghiêm trọng như bệnh tim mạch, suy thận, tổn thương thần kinh, suy giảm thị lực và các vấn đề sức khỏe lâu dài khác.

Trong những năm gần đây, cùng với sự phát triển của trí tuệ nhân tạo và học máy, các mô hình dự đoán bệnh dựa trên dữ liệu y tế ngày càng được quan tâm. Thay vì chỉ dựa hoàn toàn vào đánh giá thủ công, mô hình học máy có thể học từ dữ liệu quá khứ để tìm ra các mối quan hệ giữa những chỉ số sức khỏe của bệnh nhân và khả năng mắc bệnh.

1.2. Mục tiêu của đề tài

Đề tài "Huấn luyện mô hình AI dự đoán bệnh Diabetes" được thực hiện nhằm xây dựng và đánh giá các mô hình học máy có khả năng phân loại một người có nguy cơ mắc bệnh tiểu đường hay không dựa trên các chỉ số y tế đầu vào.

Các mục tiêu cụ thể bao gồm:

- Xử lý và phân tích bộ dữ liệu y tế về bệnh tiểu đường.
- So sánh ảnh hưởng của các phương pháp tiền xử lý dữ liệu khác nhau.
- Huấn luyện các mô hình học máy: KNN, Logistic Regression, Decision Tree.
- Tối ưu siêu tham số bằng GridSearchCV và đánh giá mô hình khách quan.
- Lựa chọn mô hình có hiệu suất tốt nhất cho bài toán phân loại nhị phân.

1.3. Phạm vi và đối tượng nghiên cứu

Bộ dữ liệu sử dụng gồm 768 mẫu với 8 thuộc tính đầu vào: số lần mang thai, nồng độ glucose, huyết áp, độ dày da, insulin, chỉ số BMI, chỉ số di truyền bệnh tiểu đường và độ tuổi. Biến đầu ra là Outcome với giá trị 0 (không mắc bệnh) và 1 (mắc bệnh).

1.4. Phương pháp thực hiện

Đề tài sử dụng phương pháp học có giám sát (Supervised Learning) với các thuật toán phân loại nhị phân, kết hợp kỹ thuật kiểm định chéo (Cross Validation) và tối ưu siêu tham số (GridSearchCV) để đánh giá và lựa chọn mô hình tốt nhất.

1.5. Cấu trúc báo cáo

- Chương I: Giới thiệu đề tài, mục tiêu và phạm vi nghiên cứu.
- Chương II: Cơ sở lý thuyết về trí tuệ nhân tạo, học máy và các thuật toán sử dụng.
- Chương III: Phân tích và thiết kế hệ thống.
- Chương IV: Triển khai hệ thống.
- Chương V: Thực nghiệm và đánh giá kết quả.
- Chương VI: Kết luận và hướng phát triển.

CHƯƠNG II. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về trí tuệ nhân tạo và học máy

Trí tuệ nhân tạo (Artificial Intelligence – AI) là lĩnh vực nghiên cứu nhằm xây dựng các hệ thống máy tính có khả năng mô phỏng một số hoạt động trí tuệ của con người như học tập, suy luận, nhận dạng, dự đoán và ra quyết định. Trong lĩnh vực y tế, AI ngày càng được ứng dụng rộng rãi để hỗ trợ phân tích dữ liệu bệnh nhân, dự đoán nguy cơ mắc bệnh và hỗ trợ bác sĩ trong quá trình chẩn đoán.

Học máy (Machine Learning) là một nhánh quan trọng của AI, cho phép máy tính học từ dữ liệu thay vì phải lập trình thủ công toàn bộ quy tắc xử lý. Thông qua quá trình huấn luyện, mô hình học máy có thể phát hiện các mẫu, mối quan hệ hoặc quy luật tiềm ẩn trong dữ liệu, từ đó đưa ra dự đoán cho các dữ liệu mới.

Trong đề tài này, học máy được sử dụng để xây dựng mô hình dự đoán một người có mắc bệnh tiểu đường hay không dựa trên các chỉ số sức khỏe như nồng độ glucose, huyết áp, insulin, BMI, tuổi và một số thông tin y tế khác.

2.2. Bài toán phân loại trong học máy

Bài toán dự đoán bệnh tiểu đường trong đề tài thuộc nhóm bài toán phân loại nhị phân (Binary Classification). Mô hình học máy nhận vào một tập các đặc trưng đầu vào và dự đoán nhãn đầu ra thuộc một trong hai lớp cho trước.

Giá trị (Outcome)	Ý nghĩa
0	Không mắc bệnh tiểu đường
1	Mắc bệnh tiểu đường

2.3. Bộ dữ liệu sử dụng trong đề tài

Bộ dữ liệu sử dụng gồm các thuộc tính y tế có liên quan đến nguy cơ mắc bệnh. Mỗi dòng biểu diễn thông tin của một cá nhân, trong đó các cột đầu vào là các đặc trưng sức khỏe, còn cột Outcome là kết quả phân loại.

Thuộc tính	Ý nghĩa
Pregnancies	Số lần mang thai
Glucose	Nồng độ glucose trong máu

Thuộc tính	Ý nghĩa
BloodPressure	Huyết áp
SkinThickness	Độ dày nếp gấp da
Insulin	Nồng độ insulin
BMI	Chỉ số khối cơ thể
DiabetesPedigreeFunction	Chỉ số di truyền liên quan đến tiểu đường
Age	Tuổi
Outcome	Kết quả: mắc bệnh hoặc không mắc bệnh

Trong đó, Outcome là biến mục tiêu cần dự đoán. Các thuộc tính còn lại được sử dụng làm đặc trưng đầu vào cho mô hình học máy.

2.4. Tiền xử lý dữ liệu

Trong bộ dữ liệu diabetes, một số cột như Glucose, BloodPressure, SkinThickness, Insulin và BMI có xuất hiện giá trị bằng 0. Về mặt y học, các giá trị này thường không hợp lý nên được xem là dữ liệu thiếu hoặc không hợp lệ. Đề tài sử dụng các hướng xử lý dữ liệu khác nhau để so sánh ảnh hưởng của tiền xử lý dữ liệu đến kết quả huấn luyện mô hình.

2.4.1. Xử lý giá trị thiếu bằng trung vị

Phương pháp thay thế giá trị thiếu (xử lý missing value) bằng trung vị của toàn bộ cột tương ứng (file clean_diabetes.csv). Trung vị ít bị ảnh hưởng bởi các giá trị ngoại lai nên thường được sử dụng trong các bộ dữ liệu y tế. Tuy nhiên, phương pháp này chưa xét đến sự khác biệt giữa nhóm mắc bệnh và nhóm không mắc bệnh.

2.4.2. Xử lý giá trị thiếu theo nhóm Outcome

Dữ liệu được chia thành hai nhóm (Outcome = 0 và Outcome = 1), sau đó các giá trị thiếu ở mỗi nhóm được thay thế bằng trung vị tương ứng của nhóm đó (file diabetes_optimal.csv). Cách làm này giúp giá trị điền vào phù hợp hơn với đặc điểm từng nhóm bệnh nhân.

2.5. Chuẩn hoá dữ liệu (StandardScaler)

Do các đặc trưng trong bộ dữ liệu có thang đo khác nhau, đề tài sử dụng phương pháp StandardScaler để chuẩn hóa, biến đổi dữ liệu sao cho mỗi đặc trưng có giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1.

Ký hiệu	Ý nghĩa
x	Giá trị ban đầu
μ	Giá trị trung bình của đặc trưng
σ	Độ lệch chuẩn của đặc trưng
z	Giá trị sau khi chuẩn hoá

2.6. Thuật toán K-Nearest Neighbors (KNN)

KNN là thuật toán học máy đơn giản và phổ biến trong bài toán phân loại. Ý tưởng chính là dự đoán nhãn của một điểm dữ liệu mới dựa trên nhãn của k điểm dữ liệu gần nó nhất trong tập huấn luyện. Khoảng cách thường sử dụng là khoảng cách Euclidean. Thuật toán nhạy cảm với thang đo dữ liệu nên cần chuẩn hóa trước khi huấn luyện.

2.7. Thuật toán Logistic Regression

Logistic Regression là thuật toán thường được sử dụng trong bài toán phân loại nhị phân. Thuật toán sử dụng hàm sigmoid để biến đổi đầu ra tuyến tính thành xác suất trong khoảng $[0, 1]$. Nếu xác suất dự đoán lớn hơn hoặc bằng 0.5, mô hình dự đoán bệnh nhân mắc bệnh; ngược lại thì không mắc bệnh.

2.8. Thuật toán Decision Tree

Decision Tree (cây quyết định) là thuật toán mô phỏng quá trình ra quyết định theo dạng cây. Mỗi nút biểu diễn một điều kiện kiểm tra trên một thuộc tính, mỗi nhánh là kết quả của điều kiện, và nút lá là nhãn dự đoán. Nếu cây quá sâu, mô hình có thể bị overfitting nên cần điều chỉnh các tham số như độ sâu tối đa.

2.9. Tối ưu tham số bằng GridSearchCV

GridSearchCV hoạt động bằng cách thử nhiều tổ hợp siêu tham số khác nhau, sau đó đánh giá từng tổ hợp bằng phương pháp kiểm định chéo (Cross Validation). Tổ hợp tham số có kết quả tốt nhất sẽ được chọn để huấn luyện mô hình cuối cùng.

Mô hình	Siêu tham số
KNN	n_neighbors, weights, metric

Mô hình	Siêu tham số
Logistic Regression	C, solver, max_iter
Decision Tree	criterion, max_depth, min_samples_split, min_samples_leaf

2.10. Các chỉ số đánh giá mô hình

Chỉ số	Ý nghĩa
Accuracy	Tỷ lệ dự đoán đúng trên tổng số mẫu
Precision	Tỷ lệ dự đoán đúng trong số các mẫu được dự đoán mắc bệnh
Recall	Tỷ lệ phát hiện đúng trong số các mẫu thực sự mắc bệnh
F1-score	Trung bình điều hoà giữa Precision và Recall
Confusion Matrix	Ma trận thể hiện số lượng dự đoán đúng và sai theo từng lớp

CHƯƠNG III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Mục tiêu thiết kế hệ thống

Hệ thống được thiết kế nhằm xây dựng một quy trình huấn luyện và đánh giá mô hình học máy có khả năng dự đoán nguy cơ mắc bệnh tiểu đường dựa trên các chỉ số sức khỏe. Từ dữ liệu đầu vào ban đầu, hệ thống thực hiện các bước tiền xử lý, chuẩn hóa, chia tập, huấn luyện, tối ưu và đánh giá kết quả.

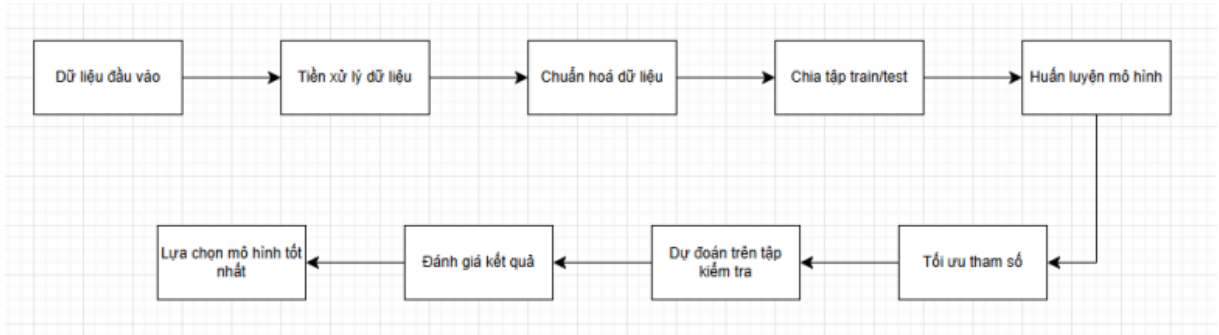
Mục tiêu chính của hệ thống bao gồm:

- Xử lý dữ liệu bệnh tiểu đường từ bộ dữ liệu gốc.
- So sánh ảnh hưởng của các phương pháp làm sạch dữ liệu khác nhau.
- Huấn luyện các mô hình học máy phục vụ bài toán phân loại nhị phân.
- Đánh giá hiệu quả của từng mô hình bằng các chỉ số đánh giá phù hợp.
- Lựa chọn mô hình có kết quả dự đoán tốt nhất.

3.2. Kiến trúc tổng quát của hệ thống

Hệ thống được thiết kế theo quy trình học máy gồm nhiều giai đoạn liên tiếp. Mỗi giai đoạn đảm nhiệm một chức năng riêng, từ xử lý dữ liệu ban đầu cho đến đánh giá kết quả mô hình.

Quy trình tổng quát của hệ thống có thể mô tả như sau:



Thành phần	Chức năng
Dữ liệu đầu vào	Chứa thông tin sức khỏe của bệnh nhân
Tiền xử lý dữ liệu	Xử lý giá trị thiếu, dữ liệu không hợp lệ
Chuẩn hoá dữ liệu	Đưa các đặc trưng về cùng thang đo
Chia dữ liệu	Tách dữ liệu thành tập huấn luyện và tập kiểm tra
Huấn luyện mô hình	Xây dựng mô hình dự đoán bệnh
Tối ưu tham số	Tìm bộ tham số tốt nhất cho từng mô hình

Thành phần	Chức năng
Đánh giá mô hình	Kiểm tra độ chính xác và hiệu quả dự đoán
Lựa chọn mô hình	Chọn mô hình có hiệu suất tốt nhất

3.3. Thiết kế dữ liệu đầu vào

Dữ liệu đầu vào của hệ thống là bộ dữ liệu diabetes, bao gồm các chỉ số y tế có liên quan đến khả năng mắc bệnh tiểu đường. Mỗi dòng dữ liệu đại diện cho một bệnh nhân, mỗi cột dữ liệu đại diện cho một thuộc tính sức khỏe.

Bộ dữ liệu gồm các thuộc tính sau:

Thuộc tính	Vai trò	Ý nghĩa
Pregnancies	Đặc trưng đầu vào	Số lần mang thai
Glucose	Đặc trưng đầu vào	Nồng độ glucose trong máu
BloodPressure	Đặc trưng đầu vào	Huyết áp
SkinThickness	Đặc trưng đầu vào	Độ dày nếp gấp da
Insulin	Đặc trưng đầu vào	Nồng độ insulin
BMI	Đặc trưng đầu vào	Chỉ số khối cơ thể
DiabetesPedigreeFunction	Đặc trưng đầu vào	Chỉ số di truyền liên quan đến tiểu đường
Age	Đặc trưng đầu vào	Tuổi
Outcome	Nhãn đầu ra	Kết quả mắc bệnh hoặc không mắc bệnh

Trong hệ thống, các thuộc tính từ Pregnancies đến Age được sử dụng làm biến đầu vào. Thuộc tính Outcome được sử dụng làm biến mục tiêu cần dự đoán.

Có thể biểu diễn dữ liệu đầu vào như sau:

$X = [\text{Prenancies}, \text{Glucose}, \text{BloodPressure}, \text{SkinThickness}, \text{Insulin}, \text{BMI}, \text{DiabetesPedigreeFunction}, \text{Age}]$

$y = \text{Outcome}$

Trong đó:

- X là tập đặc trưng đầu vào
- y là nhãn kết quả cần dự đoán

3.4. Thiết kế các phiên bản dữ liệu

File	Mô tả
diabetes.csv	Dữ liệu gốc chưa qua xử lý, dùng để phân tích và xác định missing value
clean_diabetes.csv	Dữ liệu đã thay thế missing value bằng trung vị chung toàn bộ cột
diabetes_optimal.csv	Dữ liệu đã thay thế missing value bằng trung vị theo từng nhóm Outcome

3.5. Thiết kế chia tập dữ liệu

Tập dữ liệu	Tỷ lệ	Mục đích
Tập huấn luyện	80%	Dùng để huấn luyện mô hình
Tập kiểm tra	20%	Dùng để đánh giá mô hình trên dữ liệu chưa từng thấy

Đoạn code chia dữ liệu:

```
x_train, x_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Trong đó:

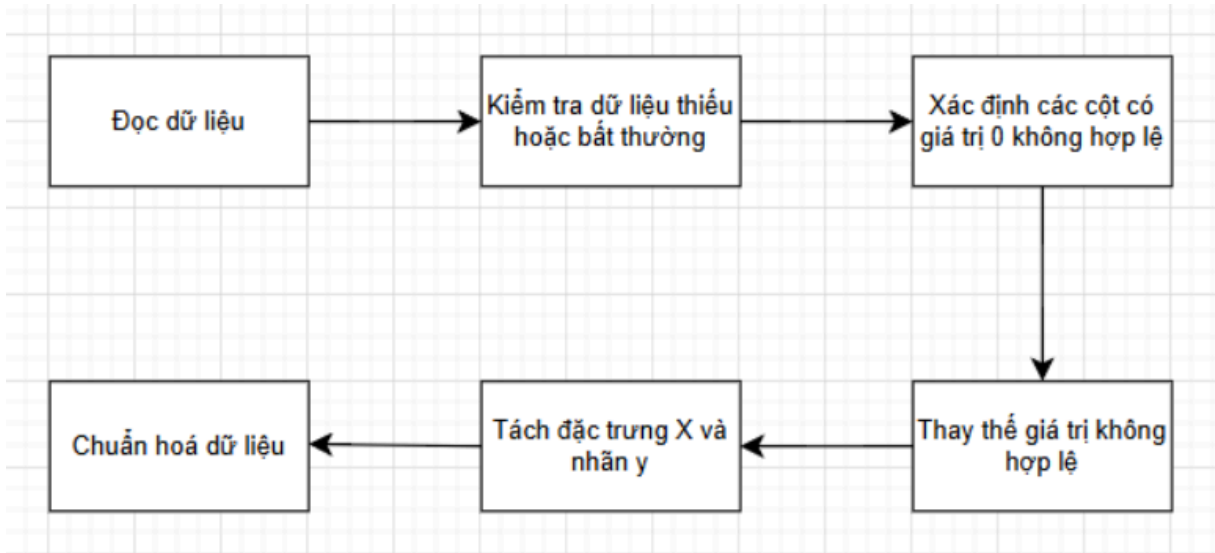
Tham số	Ý nghĩa
Test_size=0.2	20% dữ liệu được dùng làm tập kiểm tra
random_state=42	Giúp kết quả chia dữ liệu có thể lặp lại
stratify=y	Giữ tỷ lệ hai lớp Outcome = 0 và Outcome = 1 tương đối giống nhau ở tập train và test

Việc sử dụng stratify=y có ý nghĩa quan trọng trong bài toán y tế vì giúp tránh tình trạng tập huấn luyện hoặc tập kiểm tra bị lệch quá nhiều về một lớp. Nếu tỷ lệ mắc

bệnh và không mắc bệnh bị mất cân bằng quá lớn giữa hai tập, kết quả đánh giá mô hình có thể không phản ánh đúng hiệu quả thực tế.

3.6. Thiết kế quy trình tiền xử lý dữ liệu

Tiền xử lý dữ liệu là bước quan trọng nhằm cải thiện chất lượng dữ liệu trước khi đưa vào mô hình học máy. Trong hệ thống này, quá trình tiền xử lý được thực hiện qua các bước sau:



Các giá trị bằng 0 ở những cột này được xem là không hợp lệ vì không phù hợp với đặc điểm sinh học thực tế. Sau khi xác định, hệ thống tiến hành thay thế bằng các phương pháp xử lý dữ liệu tương ứng.

- Đối với file `clean_diabetes.csv`, hệ thống thay thế bằng trung vị chung của từng cột.
- Đối với file `diabetes_optimal.csv`, hệ thống thay thế bằng trung vị theo từng nhóm Outcome.

3.7. Thiết kế bước chuẩn hoá dữ liệu

Sau khi xử lý missing value, dữ liệu được chuẩn hóa trước khi đưa vào một số mô hình học máy. Nguyên nhân là các đặc trưng trong bộ dữ liệu có thang đo khác nhau. Ví dụ, Insulin có thể có giá trị rất lớn, trong khi DiabetesPedigreeFunction thường có giá trị nhỏ hơn nhiều.

Nếu không chuẩn hóa, các thuật toán dựa trên khoảng cách như KNN có thể bị ảnh hưởng bởi những thuộc tính có giá trị lớn. Vì vậy, hệ thống sử dụng phương pháp StandardScaler để chuẩn hóa dữ liệu.

Công thức chuẩn hóa:

$$z = (x - \mu) / \sigma$$

Trong đó:

Ký hiệu	Ý nghĩa
x	Giá trị ban đầu
μ	Giá trị trung bình của đặc trưng
σ	Độ lệch chuẩn của đặc trưng
z	Giá trị sau khi chuẩn hoá

Sau khi chuẩn hóa, các đặc trưng có trung bình xấp xỉ bằng 0 và độ lệch chuẩn bằng 1. Điều này giúp mô hình học ổn định hơn và hạn chế việc một đặc trưng có thang đo lớn chi phối quá trình huấn luyện.

3.8. Thiết kế chia tập dữ liệu

Để đánh giá khả năng dự đoán của mô hình trên dữ liệu chưa từng thấy, hệ thống chia dữ liệu thành hai phần:

Tập dữ liệu	Tỷ lệ	Mục đích
Tập huấn luyện	80%	Dùng để huấn luyện mô hình
Tập kiểm tra	20%	Dùng để đánh giá mô hình

Quá trình chia dữ liệu được thực hiện bằng phương pháp train_test_split. Ngoài ra, hệ thống sử dụng tham số stratify = y nhằm đảm bảo tỷ lệ giữa hai lớp Outcome = 0 và Outcome = 1 trong tập train và test được giữ tương đối giống nhau.

Việc chia dữ liệu có phân tầng giúp kết quả đánh giá đáng tin cậy hơn, đặc biệt trong bài toán phân loại bệnh, nơi số lượng mẫu giữa hai lớp có thể không hoàn toàn cân bằng.

3.9. Thiết kế mô hình huấn luyện

Trong hệ thống, ba mô hình học máy được sử dụng để huấn luyện và so sánh kết quả, bao gồm:

- K-Nearest Neighbors.
- Logistic Regression.
- Decision Tree.

Mỗi mô hình có đặc điểm riêng và phù hợp với những dạng dữ liệu khác nhau. Việc sử dụng nhiều mô hình giúp so sánh hiệu quả dự đoán và lựa chọn mô hình phù hợp nhất cho bài toán.

3.9.1. Mô hình *K-Nearest Neighbors*

KNN dự đoán nhãn của một mẫu dữ liệu mới dựa trên nhãn của các mẫu dữ liệu gần nhất trong tập huấn luyện. Vì thuật toán này dựa vào khoảng cách giữa các điểm dữ liệu nên dữ liệu cần được chuẩn hóa trước khi huấn luyện.

Trong hệ thống, KNN được sử dụng để tìm các bệnh nhân có đặc điểm sức khỏe tương đồng với mẫu cần dự đoán, từ đó đưa ra kết quả người đó có khả năng mắc bệnh hay không.

3.9.2. Mô hình *Logistic Regression*

Logistic Regression được sử dụng để dự đoán xác suất một bệnh nhân thuộc lớp mắc bệnh. Mô hình này phù hợp với bài toán phân loại nhị phân và có khả năng giải thích tương đối tốt.

Kết quả đầu ra của Logistic Regression là xác suất:

$$P(y = 1 | x)$$

Trong đó, $P(y=1|x)$ là xác suất bệnh nhân mắc bệnh tiểu đường dựa trên các đặc trưng đầu vào x . Nếu xác suất lớn hơn hoặc bằng 0.5, mô hình dự đoán bệnh nhân mắc bệnh. Ngược lại, nếu xác suất nhỏ hơn 0.5, mô hình dự đoán bệnh nhân không mắc bệnh.

3.9.3. Mô hình *Decision Tree*

Decision Tree dự đoán kết quả dựa trên hệ thống các điều kiện dạng cây. Mỗi nút trong cây kiểm tra một thuộc tính, mỗi nhánh là kết quả của điều kiện, và nút lá là nhãn dự đoán.

Mô hình này có ưu điểm là dễ hiểu, dễ biểu diễn dưới dạng luật quyết định và phù hợp để giải thích quy trình phân loại. Tuy nhiên, nếu cây quá sâu, mô hình có thể bị overfitting, do đó cần điều chỉnh tham số phù hợp.

3.10. Thiết kế tối ưu tham số

Để nâng cao hiệu quả dự đoán, hệ thống sử dụng GridSearchCV để tìm bộ siêu tham số tốt nhất cho từng mô hình. GridSearchCV hoạt động bằng cách thử nhiều tổ hợp tham số khác nhau, sau đó đánh giá từng tổ hợp bằng Cross Validation. Tổ hợp tham số có kết quả tốt nhất sẽ được chọn để huấn luyện mô hình cuối cùng.

Một số tham số có thể được tối ưu gồm:

Mô hình	Tham số cần tối ưu
KNN	n_neighbors, weights, metric
Logistic Regression	C, solver, max_iter
Decision Tree	criterion, max_depth, min_samples_split, min_samples_leaf

Việc tối ưu tham số giúp mô hình hạn chế tình trạng chọn tham số theo cảm tính và cải thiện khả năng dự đoán trên tập kiểm tra.

3.11. Thiết kế đánh giá mô hình

Sau khi huấn luyện, các mô hình được sử dụng để dự đoán trên tập kiểm tra. Kết quả dự đoán được so sánh với nhãn thực tế để đánh giá hiệu quả. Các chỉ số đánh giá được sử dụng bao gồm:

Chỉ số	Ý nghĩa
Accuracy	Tỷ lệ dự đoán đúng trên tổng số mẫu

Precision	Tỷ lệ dự đoán đúng trong số các mẫu được dự đoán mắc bệnh
Recall	Tỷ lệ phát hiện đúng trong số các mẫu thực sự mắc bệnh
F1-score	Trung bình điều hoà giữa Precision và Recall
Confusion Matrix	Ma trận thể hiện số lượng dự đoán đúng và sai theo từng lớp

Trong bài toán dự đoán bệnh, ngoài Accuracy, chỉ số Recall cũng có ý nghĩa quan trọng vì mô hình cần hạn chế bỏ sót những người thực sự mắc bệnh. Nếu một bệnh nhân mắc bệnh nhưng mô hình dự đoán là không mắc bệnh, đây là trường hợp nguy hiểm hơn so với việc dự đoán nhầm một người khỏe mạnh là có nguy cơ mắc bệnh.

Do đó, khi đánh giá hệ thống, cần xem xét đồng thời nhiều chỉ số thay vì chỉ dựa vào Accuracy.

3.12. Thiết kế đầu ra của hệ thống

Đầu ra của hệ thống gồm hai nhóm kết quả chính:

3.12.1. Kết quả dự đoán

Với một mẫu dữ liệu đầu vào gồm các chỉ số sức khỏe của bệnh nhân, hệ thống đưa ra kết quả dự đoán:

Đầu ra	Ý nghĩa
0	Bệnh nhân được dự đoán không mắc bệnh tiểu đường
1	Bệnh nhân được dự đoán mắc bệnh tiểu đường

Đối với Logistic Regression, hệ thống có thể trả thêm xác suất dự đoán. Ví dụ, $P(y=1|x) = 0.82$ có nghĩa là mô hình dự đoán bệnh nhân có 82% khả năng thuộc nhóm mắc bệnh tiểu đường.

3.12.2. Kết quả đánh giá mô hình

Sau khi chạy thực nghiệm, hệ thống trả về các kết quả đánh giá như:

- Độ chính xác của từng mô hình.
- Báo cáo phân loại gồm Precision, Recall, F1-score.
- Ma trận nhầm lẫn (Confusion Matrix).
- Bộ tham số tối ưu của từng mô hình.
- So sánh kết quả giữa các mô hình.

Từ các kết quả này, người thực hiện có thể chọn ra mô hình phù hợp nhất cho bài toán dự đoán bệnh tiểu đường.

CHƯƠNG IV. TRIỂN KHAI HỆ THỐNG

4.1. Môi trường triển khai

Hệ thống được triển khai bằng ngôn ngữ lập trình Python trên môi trường Notebook. Đây là môi trường phù hợp cho các bài toán học máy vì hỗ trợ tốt việc xử lý dữ liệu, huấn luyện mô hình, hiển thị kết quả và đánh giá mô hình theo từng bước.

Thư viện	Chức năng
pandas	Đọc, xử lý và phân tích dữ liệu dạng bảng
numpy	Hỗ trợ tính toán số học và xử lý mảng
scikit-learn	Cung cấp các thuật toán học máy, chia dữ liệu, chuẩn hóa và đánh giá
StandardScaler	Chuẩn hóa dữ liệu đầu vào
train_test_split	Chia dữ liệu thành tập huấn luyện và tập kiểm tra
accuracy_score, classification_report, confusion_matrix	Đánh giá kết quả dự đoán
GridSearchCV	Tối ưu siêu tham số cho mô hình
KNeighborsClassifier	Xây dựng mô hình KNN
LogisticRegression	Xây dựng mô hình Logistic Regression
DecisionTreeClassifier	Xây dựng mô hình Decision Tree

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
```

4.2. Đọc và chia tập dữ liệu

Hệ thống sử dụng file diabetes_optimal.csv và clean_diabetes.csv để huấn luyện mô hình. Cột Outcome được tách riêng làm biến mục tiêu (y), các cột còn lại

làm đặc trưng đầu vào (X). Dữ liệu được chia theo tỷ lệ 80/20 với stratify=y để đảm bảo tỷ lệ hai lớp cân bằng ở cả tập train và test.

Đoạn code đọc dữ liệu:

```
data = pd.read_csv(r"D:\University\Nhap_mon_tri_tue_nhan_tao\project\dataset\diabetes_optimal.csv")
# data = pd.read_csv(r"D:\University\Nhap_mon_tri_tue_nhan_tao\project\dataset\clean_diabetes.csv")
X = data.drop(columns=["Outcome"])
y = data["Outcome"]
```

Trong đó:

Biến	Ý nghĩa
Data	Toàn bộ dữ liệu diabetes sau khi đọc từ file CSV
X	Tập đặc trưng đầu vào
y	Nhãn đầu ra cần dự đoán

Cột Outcome được tách riêng làm biến mục tiêu. Các cột còn lại như Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction và Age được sử dụng làm dữ liệu đầu vào cho mô hình.

4.3. Chuẩn hoá dữ liệu

Một số thuật toán như KNN và Logistic Regression nhạy cảm với thang đo của dữ liệu. Trong bộ dữ liệu diabetes, các đặc trưng có phạm vi giá trị khác nhau. Ví dụ, Insulin có thể có giá trị lớn, trong khi DiabetesPedigreeFunction thường có giá trị nhỏ.

Vì vậy, hệ thống sử dụng StandardScaler để chuẩn hóa dữ liệu. Việc chuẩn hóa giúp đưa các đặc trưng về cùng một thang đo, giúp mô hình học ổn định và công bằng hơn giữa các thuộc tính.

Đoạn code chuẩn hóa:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Ở đây, `fit_transform()` được áp dụng trên tập huấn luyện để tính trung bình và độ lệch chuẩn, sau đó chuẩn hóa dữ liệu huấn luyện. Đối với tập kiểm tra, chỉ sử dụng `transform()` nhằm đảm bảo mô hình không học trước thông tin từ tập kiểm tra. Điều này giúp tránh hiện tượng rò rỉ dữ liệu từ tập kiểm tra vào quá trình huấn luyện.

4.4. Triển khai mô hình Logistic Regression

Mô hình đầu tiên được triển khai là Logistic Regression. Đây là thuật toán phù hợp cho bài toán phân loại nhị phân, trong đó đầu ra là xác suất một mẫu dữ liệu thuộc lớp 1, tức là có khả năng mắc bệnh diabetes.

Đoạn code huấn luyện Logistic Regression:

```
log_reg = LogisticRegression(max_iter=1000, random_state=42)

log_reg.fit(X_train_scaled, y_train)

y_pred_log = log_reg.predict(X_test_scaled)

print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_log))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_log))
print("Classification Report:")
print(classification_report(y_test, y_pred_log))
```

Trong đó:

Thành phần	Vai trò
<code>LogisticRegression(max_iter=1000)</code>	Khởi tạo mô hình Logistic Regression
<code>fit()</code>	Huấn luyện mô hình trên tập train
<code>predict()</code>	Dự đoán nhãn trên tập test
<code>accuracy_score()</code>	Tính độ chính xác
<code>confusion_matrix()</code>	Hiển thị ma trận nhầm lẫn
<code>classification_report()</code>	Hiển thị Precision, Recall, F1-score

Dữ liệu đưa vào Logistic Regression là dữ liệu đã được chuẩn hóa, vì mô hình này có thể bị ảnh hưởng bởi sự khác biệt thang đo giữa các đặc trưng.

4.5. Tối ưu mô hình Logistic Regression bằng GridSearchCV

Sau khi huấn luyện mô hình Logistic Regression cơ bản, hệ thống tiếp tục sử dụng `GridSearchCV` để tìm bộ siêu tham số tốt nhất.

Đoạn code tối ưu Logistic Regression:

```
param_grid_log = {
    "C": [0.01, 0.1, 1, 10, 100],
    "solver": ["liblinear", "lbfgs"],
    "penalty": ["l2"]
}

grid_log = GridSearchCV(
    LogisticRegression(max_iter=1000, random_state=42),
    param_grid_log,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid_log.fit(X_train_scaled, y_train)

print("Best Logistic Regression Score:", grid_log.best_score_)
print("Best Logistic Regression Params:", grid_log.best_params_)

best_log_model = grid_log.best_estimator_
y_pred_best_log = best_log_model.predict(X_test_scaled)

print("Test Accuracy:", accuracy_score(y_test, y_pred_best_log))
print(classification_report(y_test, y_pred_best_log))
```

Các tham số được tối ưu gồm:

Tham số	Ý nghĩa
C	Điều chỉnh mức độ regularization của mô hình
Solver	Thuật toán tối ưu được sử dụng
penalty	Loại điều chuẩn, ở đây sử dụng l2
cv=5	Sử dụng kiểm định chéo 5 lần
scoring= "accuracy"	Chọn mô hình tốt nhất dựa trên độ chính xác
n_jobs = -1	Sử dụng toàn bộ tài nguyên CPU để tăng tốc quá trình tìm kiếm

Sau khi GridSearchCV hoàn thành, mô hình tốt nhất được lấy ra bằng:

```
best_log_model = grid_log.best_estimator_
```

Mô hình này sau đó được dùng để dự đoán trên tập kiểm tra.

4.6. Triển khai mô hình Decision Tree

Mô hình thứ hai được triển khai là Decision Tree. Đây là thuật toán phân loại hoạt động dựa trên các luật quyết định dạng cây.

Khác với KNN và Logistic Regression, Decision Tree không yêu cầu chuẩn hóa dữ liệu vì mô hình hoạt động dựa trên việc chia nhánh theo điều kiện của từng thuộc tính.

Đoạn code huấn luyện Decision Tree:

```
dt_model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    random_state=42
)

dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_dt))
print("Classification Report:")
print(classification_report(y_test, y_pred_dt))
```

Trong đó:

Thành phần	Ý nghĩa
criterion="gini"	Sử dụng chỉ số Gini để đánh giá cách chia dữ liệu
max_depth=4	Giới hạn độ sâu tối đa của cây
random_state=42	Cố định kết quả huấn luyện
fit()	Huấn luyện mô hình
predict()	Dự đoán kết quả trên tập test

Việc giới hạn max_depth=4 giúp giảm nguy cơ overfitting, tức là tránh việc cây học quá chi tiết dữ liệu huấn luyện và dự đoán kém trên dữ liệu mới.

4.7. Tối ưu mô hình Decision Tree bằng GridSearchCV

Sau khi huấn luyện mô hình Decision Tree ban đầu, hệ thống tiếp tục tối ưu các tham số để tìm mô hình có hiệu quả tốt hơn.

Đoạn code tối ưu Decision Tree:

```
param_grid_dt = {
    "criterion": ["gini", "entropy"],
    "max_depth": [2, 3, 4, 5, 6, 7, 8, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4, 6]
}

grid_dt = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid_dt,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid_dt.fit(X_train, y_train)

print("Best Decision Tree Score:", grid_dt.best_score_)
print("Best Decision Tree Params:", grid_dt.best_params_)

best_dt_model = grid_dt.best_estimator_
y_pred_best_dt = best_dt_model.predict(X_test)

print("Test Accuracy:", accuracy_score(y_test, y_pred_best_dt))
print(classification_report(y_test, y_pred_best_dt))
```

Các tham số được tối ưu gồm:

Thành phần	Ý nghĩa
Criterion	Tiêu chí đánh giá chất lượng chia nhánh
max_depth	Độ sâu tối đa của cây
min_samples_split	Số mẫu tối thiểu để tiếp tục chia một nút
min_samples_leaf	Số mẫu tối thiểu tại một nút lá

Việc tối ưu các tham số này giúp cây quyết định đạt được sự cân bằng giữa khả năng học dữ liệu và khả năng tổng quát hóa trên dữ liệu mới.

4.8. Triển khai mô hình KNN

Mô hình thứ ba được triển khai là K-Nearest Neighbors, viết tắt là KNN. Đây là thuật toán phân loại dựa trên khoảng cách giữa các điểm dữ liệu.

Do KNN phụ thuộc trực tiếp vào khoảng cách, dữ liệu đầu vào cần được chuẩn hóa trước khi huấn luyện.

Đoạn code huấn luyện KNN:

```
knn = KNeighborsClassifier(
    n_neighbors=5,
    metric="minkowski"
)

knn.fit(X_train_scaled, y_train)

y_pred = knn.predict(X_test_scaled)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))
```

Trong đó:

Thành phần	Ý nghĩa
n_neighbors=5	Số lượng láng giềng gần nhất được xét
metric="minkowski"	Cách tính khoảng cách
fit()	Lưu dữ liệu huấn luyện để phục vụ quá trình dự đoán
predict()	Dự đoán nhãn cho dữ liệu kiểm tra

KNN không xây dựng mô hình theo dạng hàm như Logistic Regression mà lưu lại dữ liệu huấn luyện. Khi có mẫu mới, thuật toán sẽ tìm các mẫu gần nhất trong tập train để đưa ra dự đoán.

4.9. Tối ưu mô hình KNN bằng GridSearchCV

Để cải thiện hiệu quả mô hình KNN, hệ thống sử dụng GridSearchCV để tìm giá trị tốt nhất cho các tham số như số lượng láng giềng, cách tính trọng số và khoảng cách.

Đoạn code tối ưu KNN:

```
param_grid = {
    "n_neighbors": np.arange(1, 31),
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "manhattan"]
}

grid = GridSearchCV(
    KNeighborsClassifier(),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train_scaled, y_train)

best_knn = grid.best_estimator_

print("Best CV Accuracy:", grid.best_score_)
print("Best Parameters:", grid.best_params_)

y_pred_best = best_knn.predict(X_test_scaled)

print("Test Accuracy:", accuracy_score(y_test, y_pred_best))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_best))
```

Các tham số được tối ưu gồm:

Thành phần	Ý nghĩa
n_neighbors	Số lượng hàng xóm gần nhất, xét từ 1 đến 30
Weights	Cách gán trọng số cho các hàng xóm
Metric	Cách tính khoảng cách, gồm Euclidean và Manhattan
cv=5	Sử dụng kiểm định chéo 5 lần

Sau khi tìm kiếm hoàn tất, mô hình KNN tốt nhất được lưu vào biến:

```
best_knn = grid.best_estimator_
```

Mô hình này sau đó được dùng để dự đoán trên tập kiểm tra.

4.10. Triển khai mô hình KNN

Sau khi đã huấn luyện và tối ưu ba mô hình, hệ thống tiến hành tổng hợp kết quả để so sánh hiệu suất giữa các mô hình.

Đoạn code tổng hợp kết quả:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

models_result = {
    "KNN": y_pred_best,
    "Logistic Regression": y_pred_best_log,
    "Decision Tree": y_pred_best_dt
}

for name, y_pred in models_result.items():
    print("Model:", name)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred))
    print("Recall:", recall_score(y_test, y_pred))
    print("F1-score:", f1_score(y_test, y_pred))
    print("-" * 40)
```

Các chỉ số được sử dụng để so sánh gồm:

Chỉ số	Ý nghĩa
Accuracy	Tỷ lệ dự đoán đúng trên toàn bộ tập kiểm tra
Precision	Tỷ lệ dự đoán đúng trong số các mẫu được dự đoán là mắc bệnh
Recall	Tỷ lệ phát hiện đúng trong số các mẫu thực sự mắc bệnh
F1-score	Trung bình điều hòa giữa Precision và Recall

Trong bài toán dự đoán bệnh, không nên chỉ đánh giá mô hình bằng Accuracy. Chỉ số Recall cũng rất quan trọng vì nó cho biết mô hình phát hiện được bao nhiêu người thực sự mắc bệnh. Nếu Recall thấp, mô hình có thể bỏ sót nhiều trường hợp mắc bệnh, điều này không phù hợp với bài toán y tế.

CHƯƠNG V. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

5.1. Mục tiêu thực nghiệm

Kiểm tra hiệu quả của các mô hình sau khi huấn luyện, so sánh kết quả giữa KNN, Logistic Regression và Decision Tree trên hai bộ dữ liệu `clean_diabetes.csv` và `diabetes_optimal.csv`, từ đó lựa chọn mô hình tốt nhất cho bài toán dự đoán bệnh tiểu đường (diabetes).

5.2. Kết quả huấn luyện mô hình Logistic Regression

Bộ dữ liệu	Không sử dụng GridSearchCV	Sử dụng GridSearchCV
<code>clean_diabetes.csv</code>	0.7078	0.7078
<code>diabetes_optimal.csv</code>	0.7078	0.7078

5.3. Kết quả huấn luyện mô hình Decision Tree

Bộ dữ liệu	Không sử dụng GridSearchCV	Sử dụng GridSearchCV
<code>clean_diabetes.csv</code>	0.7857	0.6883
<code>diabetes_optimal.csv</code>	0.8377	0.8446

5.4. Kết quả huấn luyện mô hình KNN

Bộ dữ liệu	Không sử dụng GridSearchCV	Sử dụng GridSearchCV
<code>clean_diabetes.csv</code>	0.7532	0.7597
<code>diabetes_optimal.csv</code>	0.8117	0.8506

5.5. Bảng so sánh kết quả các mô hình

5.5.1. Bộ dữ liệu `clean_diabetes.csv`

Mô hình	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.7078	0.6000	0.5000	0.5454
KNN	0.7597	0.6977	0.5556	0.6186
Decision Tree	0.6883	0.6364	0.2593	0.3684

5.5.2. Bộ dữ liệu `diabetes_optimal.csv`

Mô hình	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.7078	0.5882	0.5556	0.5714
KNN	0.8506	0.8039	0.7593	0.7810
Decision Tree	0.8446	0.7679	0.7963	0.7818

Kết luận: Từ bảng kết quả trên, có thể thấy mô hình KNN đạt kết quả tốt nhất trên các tập kiểm tra

5.6. Nhận xét kết quả thực nghiệm

- Bộ dữ liệu diabetes_optimal.csv cho kết quả tốt hơn clean_diabetes.csv ở tất cả các mô hình.
- Việc xử lý missing value theo nhóm Outcome giúp dữ liệu phản ánh tốt hơn đặc điểm từng nhóm bệnh nhân.
- GridSearchCV giúp cải thiện kết quả một số mô hình, đặc biệt là KNN và Decision Tree.
- KNN đạt kết quả tốt nhất với Accuracy = 0.8506, Precision = 0.8039, Recall = 0.7593, F1-score = 0.7810.
- Trong bài toán y tế, không chỉ quan tâm Accuracy mà còn phải chú ý Recall để hạn chế bỏ sót người mắc bệnh.

CHƯƠNG VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đề tài đã xây dựng thành công một hệ thống dự đoán bệnh tiểu đường sử dụng các thuật toán học máy. Qua quá trình thực nghiệm, kết quả cho thấy:

- Phương pháp xử lý missing value theo nhóm Outcome mang lại chất lượng dữ liệu tốt hơn và cải thiện hiệu suất mô hình đáng kể.
- Mô hình KNN với GridSearchCV đạt hiệu suất tốt nhất trên bộ dữ liệu diabetes_optimal.csv (Accuracy: 85.06%).
- GridSearchCV và Cross Validation là công cụ hiệu quả để lựa chọn siêu tham số tối ưu, tránh overfitting.
- Trong bài toán y tế, việc đánh giá mô hình cần xem xét toàn diện nhiều chỉ số, đặc biệt là Recall.

6.2. Hướng phát triển trong tương lai

- Mở rộng bộ dữ liệu với nhiều mẫu hơn để cải thiện khả năng tổng quát hóa của mô hình.
- Thử nghiệm các thuật toán học máy nâng cao như Random Forest, XGBoost, SVM.
- Áp dụng kỹ thuật xử lý mất cân bằng lớp (SMOTE, class weighting) để cải thiện Recall.
- Xây dựng giao diện ứng dụng cho phép người dùng nhập thông tin sức khỏe và nhận kết quả dự đoán.
- Triển khai mô hình trên môi trường web hoặc di động để ứng dụng thực tế trong y tế.

TÀI LIỆU THAM KHẢO

- [1] Smith, J.W., Everhart, J.E., Dickson, W.C., Knowler, W.C., Johannes, R.S. (1988). Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus. Proceedings of the Symposium on Computer Applications and Medical Care.
- [2] Scikit-learn Documentation. <https://scikit-learn.org/stable/>.
- [3] Kaggle - Pima Indians Diabetes Database. <https://www.kaggle.com/datasets/salihacur/diabetes>.